Java Silver（SE 17・1Z0-825-JPN）　1 Java の概要と簡単なJavaプログラムの作成　#01

# プログラムの実行とパッケージ

.java から .class へ、JVM で動くまで ― main メソッドとコマンドライン引数、javac と java の打ち方、クラスパス、ソースファイルモード、パッケージ宣言とインポート

各ページの下に、根拠にした Oracle・OpenJDK の資料を書いた（青い文字を押すと開く）。コードは Claude が作り、JDK 17 で動かして確かめた

# 全体像 試験内容チェックリスト 1

**1 Java のしくみ**
ソースファイル・javac・クラスファイル・バイトコード・JVM・JDK

**2 main メソッドとコマンドライン引数**
public static void main(String[] args)、起動できない形、args の中身

**3 コンパイルと実行**
javac・java、-d、クラスパス(-cp)、完全修飾名、ソースファイルモード

**4 パッケージとソースファイルの決まり**
package 宣言、並び順、public クラスとファイル名

**5 インポート**
単一型・オンデマンド・static、java.lang の自動インポート、名前のぶつかり

**Hello が表示されるまで(Claude が作成)**




JDK 11 からは java Hello.java で 1 回にまとめられる(第3章)

**芯:**javac には**ファイル名**、java には**クラス名**を渡す。

出典:Oracle「Java SE 17 Programmer I(1Z0-825-JPN)」試験内容チェックリスト(mainメソッドを持つ実行可能なJavaプログラムの作成／コマンドラインでのJavaプログラムのコンパイルと実行／パッケージの使用)／javac ドキュメント(JDK 17)(compiles them into class files that run on the Java Virtual Machine)

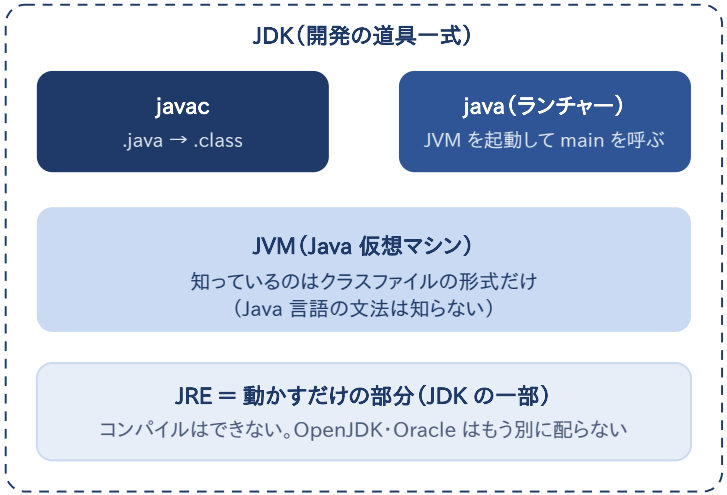
第1章

# Java のしくみ

CPU ではなく、JVM のために翻訳する。

# ソース・クラスファイル・JVM・JDK

**道具とファイルの関係（Claude が作成）**




| もの | 中身・役目 |
|---|---|
| .java | ソースファイル。人が Java の文法で書く。javac は拡張子 .java しか受け付けない |
| .class | クラスファイル。**バイトコード**（JVM の命令）と記号表。**クラスごとに 1 つ**できる |
| JVM | クラスファイルを動かす仮想の計算機。OS・CPU のちがいを吸収する |
| JDK | javac・java などの道具一式。「Java を入れる」＝JDK を入れる |

**まちがえやすい：**バイトコードは CPU の**機械語ではない**。JVM は「必ずインタプリタ」でもない（CPU の命令に翻訳して動かしてもよい）。

出典：JVMS（Java SE 17）1.2（The Java Virtual Machine knows nothing of the Java programming language, only of a particular binary format, the class file format／It is not inherently interpreted）／dev.java「Getting Started with Java」（The JRE … is a subset of the JDK that is not distributed by the OpenJDK or Oracle anymore／You cannot compile your code with the tools provided in the JRE）／javac ドキュメント（Source files must have a file name extension of .java）

# エラーが見つかるのは、いつか

Hello.java（Claude が作成）

```
public class Hello {
  public static void main(String[] args) {
    System.out.println(args.length + ":" + args[0]);
  }
}
```

JDK 17 で動かした結果

```
$ javac Hello.java          ← 何も出ない＝成功
$ java Hello a b c
3:a
$ java Hello
Exception in thread "main"
java.lang.ArrayIndexOutOfBoundsException:
Index 0 out of bounds for length 0
```

| いつ | 例 |
| --- | --- |
| **コンパイル時（javac）** | 文法の誤り・見つからないクラス・あいまいな名前・public クラスとファイル名の不一致。**.class ができない** |
| **起動時（java）** | メインクラスが見つからない・main の形がちがう（static がない など） |
| **実行中** | 配列の範囲外など。**例外**として知らされる（#13） |

**本番の選択肢：**「コンパイルエラー」「実行時に例外がスローされる」「〇〇と表示される」が並ぶ。**どの段階で止まるか**を先に考える。左の例は、javac は引数の個数を知らないので**コンパイルは通る**。

出典：JLS（Java SE 17）12.1（passing it a single argument which is an array of strings）／実行結果は JDK 17（Eclipse Temurin 17.0.21）で確かめたもの

第2章

# main メソッドとコマンドライン引数

public・static・void・String の配列。どれが欠けても入口にならない。

# main の形を分解する




**JLS 12.1.4:** main は public・static・void で、型が「String の配列」の引数を 1 つ持つ。`String[] args` と `String... args`（可変長引数）のどちらでもよい。

**修飾子の順番は自由:** `static public` でもよい。`final` を足してもよい。ただし**大文字の Main** は別の名前で、入口にならない（Java は大文字と小文字を区別する）。

出典：JLS 12.1.4 Invoke Test.main（The method main must be declared public, static, and void. It must specify a formal parameter whose declared type is array of String／public static void main(String... args)）／java ドキュメント（JDK 17） Description（it must not return any value, and it must accept a String array as a parameter）

# 起動できる main・できない main

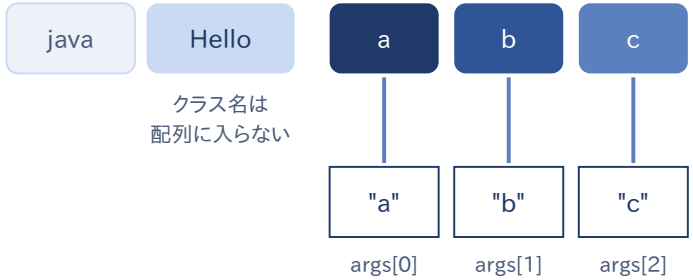
| A.java の main の宣言 | javac | java A | JDK 17 の表示・理由 |
|---|---|---|---|
| `public static void main(String[] args)` | OK | **起動** | 標準の形 |
| `static public void main(String args[])` | OK | **起動** | 順番と [] の位置は自由 |
| `public static final void main(final String... a)` | OK | **起動** | final・可変長引数・名前の変更は OK |
| `public void main(String[] args)` | OK | **失敗** | Main method is not static in class A |
| `static void main(String[] args)` | OK | **失敗** | Main method not found in class A（public がない） |
| `public static int main(String[] args)` | OK※ | **失敗** | Main method must return a value of type void |
| `public static void main(String args)` | OK | **失敗** | Main method not found（配列でない） |
| `public static void Main(String[] args)` | OK | **失敗** | Main method not found（名前がちがう） |

**ポイント：**形のちがう main も**ふつうのメソッドとしては正しい**ので、コンパイルは通る。止まるのは **java で起動したとき**。※ int を返すなら `return 0;` が必要（ないと missing return statement）。

出典：JLS 12.1.4 ／ java ドキュメント Description（The method must be declared public and static, it must not return any value）／表示は JDK 17（Temurin 17.0.21）で 1 つずつ確かめたもの

# コマンドライン引数は args に入る

**java Hello a b c のとき（Claude が作成）**




**args.length は 3**

| 打ったコマンド（bash） | args の中身 |
|---|---|
| `java Hello a b c` | "a"・"b"・"c"（3 つ） |
| `java Hello "a b" c` | "a b"・"c"（2 つ。引用符はシェルが外す） |
| `java Hello 10 20` | "10"・"20"（数字も**String**） |
| `java Hello` | **長さ 0 の配列**（null ではない） |

> **C 言語とのちがい：**argv[0] にプログラム名が入る C とちがい、Java の args[0] は**最初の引数**。JLS 12.1 の例 `java Test reboot Bob Dot Enzo` では、配列は 4 つの文字列。

出典：JLS 12.1（java Test reboot Bob Dot Enzo will typically start a Java Virtual Machine by invoking method main of class Test … passing it an array containing the four strings）／java ドキュメント Synopsis（Arguments following mainclass … are passed as arguments to the main class）／表の結果は JDK 17 で確かめたもの

第3章

# コンパイルと実行

javac にはファイル、java にはクラス。パッケージはディレクトリになる。

# javac と java の打ち方

```
$ javac Hello.java
$ ls
Hello.class  Hello.java
$ java Hello a b c
3:a
$ java Hello.class
Error: Could not find or load main class Hello.class
Caused by: java.lang.ClassNotFoundException: Hello.class
$ javac Hello
error: Class names, 'Hello', are only accepted
if annotation processing is explicitly requested
$ java hello
Error: Could not find or load main class hello
```

JDK 17 で打った結果（Claude が作成した Hello.java）

| 道具 | 渡すもの |
|---|---|
| javac | **ファイル名**（.java まで書く）。いくつでも並べられる |
| java | **クラス名**（.class は付けない）。パッケージがあれば**完全修飾名** |

**Hello.class がだめな理由：**java は最初の引数を「完全修飾名」として読む。「.」はパッケージの区切りなので、Hello.class は「Hello パッケージの class クラス」を探す意味になり、見つからない。

**大文字・小文字：**クラス名の Hello と hello は別物。

出典：java ドキュメント（java [options] mainclass [args ...]／the first argument that isn't an option of the java command is the fully qualified name of the class to be called）／javac ドキュメント Description（Source files must have a file name extension of .java. Class files have a file name extension of .class）

# パッケージはディレクトリになる

src/com/example/app/Main.java（Claude が作成）

```
package com.example.app;

import com.example.util.Greeter;

public class Main {
  public static void main(String[] args) {
    System.out.println(Greeter.hello("Java"));
  }
}
```

```
$ javac -d out src/com/example/util/Greeter.java \
                src/com/example/app/Main.java
$ java -cp out com.example.app.Main
Hello, Java
$ java -cp out Main
Error: Could not find or load main class Main
$ cd out/com/example/app; java Main
… NoClassDefFoundError: Main (wrong name: com/example/app/Main)
```

**-d out で作られる階層**

```
out/
└── com/
    └── example/
        ├── app/
        │   └── Main.class      ← com.example.app.Main
        └── util/
            └── Greeter.class   ← com.example.util.Greeter
```

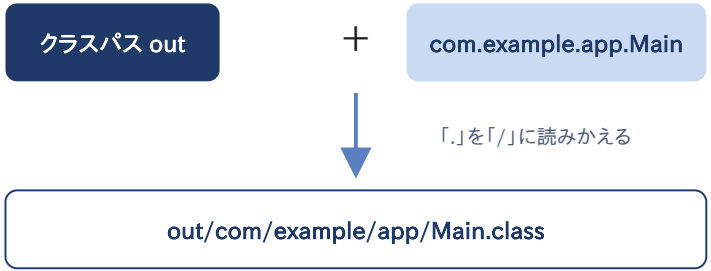
**-d:** クラスファイルの置き場所。パッケージ名どおりのディレクトリを**自動で作る**。-d がなければ、ソースファイルと**同じディレクトリ**にできる。

**実行:** クラスパスには**パッケージの根元**（out）を書き、クラスは**完全修飾名**で書く。

出典：javac ドキュメント -d（javac puts the class file in a subdirectory that reflects the … package name／If the -d option is not specified, then javac puts each class file in the same directory as the source file）／Arrangement of Source Code（The package p.q … is represented as the subdirectory q of directory p）／結果は JDK 17 で確かめたもの

# クラスパス ― クラスを探す場所

**java -cp out com.example.app.Main の探し方**




-cp も CLASSPATH もなければ、クラスパスは「.」（カレントディレクトリ）

| 決め方 | 中身 |
|---|---|
| -cp・-classpath・--class-path | コマンドで指定。**CLASSPATH より優先** |
| 環境変数 CLASSPATH | -cp がないとき使う。javac ドキュメントは「設定せず -cp で明示」を勧める |
| どちらもない | **カレントディレクトリ（.）** |

| OS | 2 つ以上並べるときの区切り |
|---|---|
| Linux・macOS | :（例 `-cp out:lib/util.jar`） |
| Windows | ;（例 `-cp out;lib\util.jar`） |

> **JAR もクラスパスに書ける。**ディレクトリ名のあとに * を書くと、その中の .jar をすべて並べたのと同じ（java ドキュメント）。

出典：java ドキュメント --class-path（Specifying classpath overrides any setting of the CLASSPATH environment variable. If the class path option isn't used and classpath isn't set, then the user class path consists of the current directory (.)／a class path element that contains a base name of an asterisk (*) is considered equivalent to specifying a list of all the files in the directory with the extension .jar）／javac ドキュメント（semicolon ; on Windows, or colon : on other systems／it is recommended that this environment variable should not be set）

# ソースファイルモード（JDK 11〜）

```
$ ls
Hello.java
$ java Hello.java p q
2:p
$ ls
Hello.java          ← .class は残らない
```

Main.java に 2 つのクラス（Claude が作成）

```
class T {
  public static void main(String[] a) { System.out.println("T"); }
}
class Main {
  public static void main(String[] a) { System.out.println("Main"); }
}
```

```
$ java Main.java
T                   ← 最初に書いたクラスが動く
```

| 決まり | 中身 |
|---|---|
| 入り方 | .java のファイルを java に渡す |
| コンパイル | **メモリの上**。クラスファイルは作らない |
| 動くクラス | ファイルの**最初**のトップレベルクラス（ファイル名と合っていなくてよい） |
| ほかのソース | **探さない**。別ファイルの B を使うと cannot find symbol |
| 引数 | ファイル名のあとの文字列が main に渡る |

**javac とのちがい：**`javac A.java` は、A が使う B.java もクラスパスから探してコンパイルし B.class を作る。ソースファイルモードは 1 ファイルだけ。

出典：java ドキュメント Using Source-File Mode（the source file is compiled into memory／No other source files are found and compiled／The class to be executed is the first top-level class found in the source file）／JEP 330: Launch Single-File Source-Code Programs（Release 11）／結果は JDK 17 で確かめたもの

第4章

# パッケージとソースファイルの決まり

package → import → クラス。public は 1 つで、ファイル名と同じ。

# ソースファイルの並び順

**1 つのソースファイル（コンパイル単位）の形**

**コメント・空白**
どこに書いてもよい（宣言ではない）

**① package 宣言（0 か 1 つ）**
なければ無名パッケージ

**② import 宣言（0 個以上）**
効くのはこのファイルの中だけ

**③ クラス・インタフェースの宣言（0 個以上）**
public なものは 1 つまで

> JLS 7.3：[PackageDeclaration] {ImportDeclaration} {TopLevelClassOrInterfaceDeclaration}。3 つとも省けるが、**順番は変えられない**。

NG の例と JDK 17 の表示

```
import java.util.*;
package p;            // import のあとに package
```

```
error: class, interface, enum, or record expected
```

```
package p;
package q;            // 2 つ目の package
```

```
error: class, interface, enum, or record expected
```

OK の例

```
/* メモ */
// コメントは package の前でもよい
package p;
import java.util.*;
public class T { }
```

出典：JLS 7.3 Compilation Units（OrdinaryCompilationUnit: [PackageDeclaration] {ImportDeclaration} {TopLevelClassOrInterfaceDeclaration}／A compilation unit that has no package declaration is part of an unnamed package）／JLS 3.2（after white space and comments are discarded）

# public クラスとファイル名・トップレベルの修飾子

| ファイルと中身 | javac の結果 | 説明 |
| --- | --- | --- |
| T.java に `public class T` | OK・T.class | public なクラスはクラス名＋.java のファイルに書く |
| Main.java に `public class T` | エラー | class T is public, should be declared in a file named T.java |
| T.java に `public class T` と `public class U` | エラー | public は 1 ファイルに 1 つまで。U でも同じエラー |
| T.java に `public class T` と `class U` | OK・T.class と U.class | public でないクラスは足してよい。main があれば `java U` も動く |
| Main.java に `class T` と `class Main` | OK・T.class と Main.class | public でなければファイル名と合っていなくてよい |
| `final class A {}`・`abstract class A {}` | OK | final・abstract はトップレベルに付けられる（#11） |
| `private`・`protected`・`static class A {}` | エラー | modifier private not allowed here（protected・static も同じ） |

> **JLS 7.6:** ファイル名の決まりは「ファイルシステムを使う処理系が課してよい制約」で、javac は守らせる。トップレベルのクラスのアクセスは **public か、何も書かない（同じパッケージだけ）** の 2 通り。

出典：JLS 7.6 Top Level Class and Interface Declarations（It is a compile-time error if a top level class or interface declaration contains any one of the following access modifiers: protected, private, or static／The class or interface is declared public … there must be at most one such class or interface per compilation unit）／結果は JDK 17 で確かめたもの

# 無名パッケージとサブパッケージ

## 無名パッケージ

```
// Util.java（package を書かない）
public class Util { }

// p/T.java
package p;
import Util;          // error: '.' expected
public class T { }
```

**名前付きパッケージから、無名パッケージの型は使えない。**インポートできるのは名前付きパッケージの型だけ（JLS 7.5.1）。import を書かずに Util と書いても cannot find symbol。無名パッケージは「小さなプログラムや学習の始め」のための便利な仕組み（JLS 7.4.2）。

## サブパッケージは別のパッケージ




import java.util.*;
左の箱だけ

import java.util.concurrent.*;
右の箱だけ

*** はサブパッケージを含まない。**`import java.util.*;` のあと `concurrent.ConcurrentHashMap` と書くと package concurrent does not exist。パッケージそのもの（`import java.util;`）もインポートできない。

出典：JLS 7.4.2 Unnamed Packages（provided … principally for convenience when developing small or temporary applications or when just beginning development）／7.5.1（The class or interface must be either a member of a named package … or a compile-time error occurs／Example 7.5.1-3 an import declaration cannot import a subpackage, only a class or interface）／結果は JDK 17 で確かめたもの

第5章

# インポート

短い名前で書くための宣言。中身を取り込むわけではない。

# インポートの 4 種類と java.lang

| 書き方 | 種類 | 使えるようになるもの |
|---|---|---|
| `import java.util.List;` | **単一型** | List という 1 つの型 |
| `import java.util.*;` | **オンデマンド** | java.util の型すべて（使った分だけ。サブパッケージは含まない） |
| `import static java.lang.Math.max;` | **単一 static** | Math の static メンバー max（同じ名前のオーバーロードすべて） |
| `import static java.lang.Math.*;` | **static オンデマンド** | Math の static メンバーすべて（`max(3, abs(-5))`・`PI`） |
| （書かない） | **自動** | **java.lang** の型すべて：String・Math・System・StringBuilder・Integer など |

**まちがえやすい書き方（JDK 17 の表示）**

```
static import java.lang.Math.max;  // 語順が逆
  → class, interface, enum, or record expected
import static java.lang.Math;      // クラス名で終わる
  → cannot find symbol
import java.lang.Math.*;           // static がない
  → max(1, 2) が cannot find symbol
```

**import の正体：**型や static メンバーを**短い名前**で書けるようにするだけ。クラスの中身はコピーしない。効くのは**書いたファイルの中だけ**で、同じパッケージのほかのファイルには効かない。import しなくても**完全修飾名**（`java.util.ArrayList`）で書けば使える。

出典：JLS 7.5（makes classes, interfaces, or members available by their simple names only within the compilation unit that actually contains the import declaration）／7.5.1〜7.5.4（import TypeName ;／import PackageOrTypeName . * ;／import static TypeName . Identifier ;／import static TypeName . * ;）／7.3（Every compilation unit implicitly imports every public class or interface declared in the predefined package java.lang）

# 名前がぶつかったとき ー Date の例

| import の組み合わせ | Date を使うと | JDK 17 の結果・理由 |
|---|---|---|
| `import java.util.*;`<br>`import java.sql.*;` | エラー | reference to Date is ambiguous。**使わなければエラーにならない** |
| `import java.util.Date;`<br>`import java.sql.*;` | java.util.Date | 単一型インポートが * のインポートを**隠す**（シャドウイング） |
| `import java.util.Date;`<br>`import java.sql.Date;` | エラー | a type with the same simple name is already defined by the single-type-import of Date。**使わなくてもエラー** |
| `import java.util.*;` と自分の `class Vector {}` | 自分の Vector | 同じファイル・同じパッケージの型が * のインポートより優先（JLS 7.6 の例） |

**強い順：**① 自分のファイル・単一型インポート　② 同じパッケージの型　③ * のインポート（java.lang も ③ と同じ扱い）。①どうし・③どうしで同じ名前が並ぶと、決められずエラーになる。

**直し方：**どちらかを単一型インポートにする。もう片方は `java.sql.Date` のように完全修飾名で書く。

出典：JLS 7.5.1（If two single-type-import declarations in the same compilation unit attempt to import classes or interfaces with the same simple name, then a compile-time error occurs）／7.5.2 Example 7.5.2-1（might be shadowed by a single-type-import declaration … by a class or interface named Vector and declared in the package to which the compilation unit belongs）／7.6 Example 7.6-1／結果は JDK 17 で確かめたもの

# 問われ方とまちがえやすい点

| こう聞かれたら | 答えの考え方 |
| --- | --- |
| **起動できる main は** | **public・static・void・String 配列**。順番・[] の位置・final・可変長・引数名は自由。形がちがってもコンパイルは通る |
| **args[0] は何か** | クラス名のあとの**最初の引数**。引数なしは長さ 0 → args[0] で例外 |
| **正しいコマンドは** | `javac Xxx.java` → `java Xxx`。パッケージなら `java -cp 根元 完全修飾名` |
| **.class はどこにできる** | -d あり → 指定先＋パッケージの階層。なし → ソースと同じ場所 |
| **java X.java の結果** | メモリでコンパイル、最初のクラスが動く、.class なし、ほかのソースは探さない |
| **ファイルの並び** | package（1 つ）→ import → クラス。前に置けるのはコメントだけ |
| **import で使えるか** | * はサブパッケージを含まない。java.lang は自動。`import static` の語順 |
| **同じ名前の型** | * どうしは**使えば**エラー、単一型どうしは**書いただけで**エラー、単一型 ＞ * |

出典：このPDFの各ページの資料

# まとめ

**しくみ**
.java →（javac）→ .class（バイトコード）→（java）→ JVM。JDK が道具一式

**main と引数**
public static void main(String[] args)。args にクラス名は入らない

**コンパイルと実行**
-d でパッケージの階層、-cp でパッケージの根元、java には完全修飾名。java X.java は JDK 11〜

**パッケージとインポート**
package → import → クラス。public は 1 つでファイル名と同じ。java.lang は自動

| エラーの表示 | 原因 |
|---|---|
| Could not find or load main class | クラス名・クラスパス・.class を付けた |
| Main method not found | public がない・名前や引数の型がちがう |
| Main method is not static | static がない |
| should be declared in a file named | public クラスとファイル名がちがう |
| reference to … is ambiguous | * のインポートどうしで名前がぶつかった |

**次は #02 変数とデータ型。**基本データ型・変数のスコープ・var（ローカル変数型推論）へ進む。

出典：このPDFの各ページの資料